# Quick Sort -

## Algorithm (Step-by-step):

- Pick a pivot element (commonly first, last, or random).
- Partition the array → elements smaller than pivot on left, larger on right.
- Recursively apply the same steps to left & right subarrays.
- Combine → sorted array forms automatically after recursion.

## Time Complexity:

- Best case: O(n log n)
- Worst case: O(n log n)
- Average case: $O(n^2)$ (when pivot choice is poor, e.g., sorted array with last element pivot)
- Space Complexity: O(log n) (Recursion Stack)

Key Point: Faster than Merge Sort in practice due to **in-place partitioning**.

# Quick Sort -

*Ques. Sort an array using  Quick Sort.*

```
def partition(arr, low, high):
    pivot = arr[high]  # choose last element as pivot
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1

def quick_sort(arr, low, high):
    if low < high:
        pi = partition(arr, low, high)
        quick_sort(arr, low, pi - 1)  # left subarray
        quick_sort(arr, pi + 1, high)  # right subarray
    return arr

arr = [10, 7, 8, 9, 1, 5]
print("Sorted Array:", quick_sort(arr, 0, len(arr)-1))
```

# Merge Sort VS Quick Sort

| Feature | Merge Sort | Quick Sort |
| --- | --- | --- |
| Approach | Divide & Conquer | Divide & Conquer |
| Time Complexity | O(n log n) (best, avg, worst) | O(n log n) avg & best, $O(n^2)$ worst |
| Space Complexity | O(n) (extra arrays) | O(log n) (in-place) |
| Stability | ✅ Stable | ❌ Not stable (by default) |
| Best Use Case | Large datasets, linked lists, external sorting | General-purpose, faster in practice for arrays |
| Drawback | Needs extra memory | Worst case $O(n^2)$ if pivot choice is poor |

# Merge Sort VS Quick Sort

Key Takeways :-

- Use **Merge Sort** when stability is required or dataset is huge.
- Use **Quick Sort** for in-memory arrays (generally faster, in-place).
- Hybrid algorithms (like **Timsort in Python's sort()**) combine strengths of both.